NAME :- Lokare Mahir Abdul Majeed

ROLL NO :- 23AI34

BATCH :- 02

EXPERIMENT NO :- 11

AIM :- To design and implement a Recurrent Neural Network (RNN) model for sequence classification and sequence-to-sequence modelling.

OBJECTIVE :-

1. To understand the concept of Recurrent Neural Networks.
2. To understand sequence learning using RNN.
3. To understand how hidden states store information from previous time steps.
4. To design an RNN model for sequence classification.
5. To train the RNN model using sequential data.
6. To evaluate the model and visualize its classification performance.

THEORY :-

1. Recurrent Neural Network :-

A Recurrent Neural Network (RNN) is a type of neural network designed to process sequential data. Unlike a traditional neural network, an RNN maintains a hidden state that carries information from previous time steps. This allows the network to use information from earlier elements of a sequence when processing the current element. RNNs are commonly used for text, speech, time series and other sequence-based applications.

For example, in a sentence, the meaning of a word can depend on the words that appeared before it. Similarly, in time series data, the current value may depend on previous observations. RNNs are designed to capture these temporal dependencies.

2. Sequential Data

Sequential data is data in which the order of elements is important. Examples include sentences, speech signals, stock prices, sensor readings and time series. A sequence can be represented as:

$$X = [x1, x2, x3, \ldots, xt]$$

where xt represents the input at time step t. The RNN processes the sequence one element at a time and updates its hidden state after every time step.

3. RNN Architecture

The basic RNN architecture consists of an input, hidden state and output. At every time step, the current input is combined with the hidden state from the previous time step. The hidden state can be represented as:

$$ht = tanh(Wx*xt + Wh*ht{-}1 + b)$$

where xt is the current input, ht−1 is the previous hidden state and ht is the current hidden state. The hidden state acts as the memory of the network and allows information from previous time steps to influence the current output.

4. Hidden State

The hidden state is one of the most important components of an RNN. It stores information about previously processed elements of the sequence. At the beginning of a sequence, the hidden state is generally initialized with zeros. As the RNN processes each input, the hidden state is updated. For example:

$$X1 \rightarrow h1;\ X2 + h1 \rightarrow h2;\ X3 + h2 \rightarrow h3$$

Thus, the information from previous time steps is carried forward through the hidden states.

5. Sequence Classification

Sequence classification is a task in which an entire sequence is assigned to a particular class. The input may contain multiple time steps, but the final output is usually a single class.

For example, in sentiment classification, a sentence can be classified as positive or negative. The RNN processes the words sequentially and uses the information stored in the final hidden state to determine the class. For multi-class classification, a Softmax output layer can be used to produce probabilities for different classes.

6. Many-to-One RNN

Sequence classification is commonly represented using a many-to-one RNN architecture. In this architecture, multiple input elements are processed sequentially, but only one final output is generated. For example:

$$x1, x2, x3, x4 \rightarrow Class$$

This structure is useful for applications such as sentiment classification, activity recognition and document classification.

7. Sequence-to-Sequence Modelling

Sequence-to-Sequence (Seq2Seq) modelling is a technique in which one sequence is converted into another sequence. The input and output sequences can have different lengths. Seq2Seq models are commonly based on an encoder-decoder architecture.

A sequence-to-sequence system can be used for language translation, text generation, speech recognition and other applications where sequential input needs to produce sequential output.

8. Encoder

The encoder processes the input sequence one element at a time. It updates its hidden state as it processes each element and creates a representation containing information about the input sequence. The encoder can be represented as:

$$Input\ Sequence \rightarrow RNN\ Encoder \rightarrow Context\ Representation$$

The context representation is then passed to the decoder.

9. Decoder

The decoder receives the information produced by the encoder and generates the output sequence step by step. The decoder uses the previous output and its current hidden state to generate the next output. The general architecture is:

$$Input\ Sequence \rightarrow Encoder \rightarrow Context\ Vector \rightarrow Decoder \rightarrow Output\ Sequence$$

In a simple RNN-based Seq2Seq model, both the encoder and decoder can use RNN layers.

10. Sequence-to-Sequence Applications

Seq2Seq models are used in applications where an input sequence needs to be transformed into another sequence. Some common applications include machine translation, text summarization, speech recognition, chatbot responses and time-series prediction. For example, in a chatbot, the user's message is treated as the input sequence and the generated response is treated as the output sequence.

11. Backpropagation Through Time

RNNs are trained using a technique called Backpropagation Through Time (BPTT). Since an RNN processes data over multiple time steps, the network can be conceptually unfolded across those time steps during training. The error is propagated backward through the sequence so that the model can update its weights and learn sequential patterns.

A major limitation of basic RNNs is that they can have difficulty learning long-term dependencies because of the vanishing gradient problem. LSTM and GRU networks were developed to handle long-term dependencies more effectively.

12. Working of RNN Sequence Classification

The working process can be summarized as follows: sequential input data is prepared; the input sequences are converted into numerical representations; the sequences are provided to the RNN one time step at a time; the RNN updates its hidden state after processing each element; the final hidden representation contains information about the sequence; the representation is passed to the Dense layer; the Softmax layer produces class probabilities; the class with the highest probability is selected as the prediction; the model compares the predicted class with the actual class; the weights are updated using backpropagation through time.

PROCEDURE :-

1. Import TensorFlow, NumPy and Matplotlib.
2. Create a small sequential classification dataset.
3. Convert the sequences into suitable input format.
4. Create the RNN model.
5. Add an Embedding layer.

6. Add a SimpleRNN layer.
7. Add a Dense output layer with Softmax activation.
8. Compile the model using Adam optimizer and sparse categorical cross-entropy.
9. Train the RNN model.
10. Evaluate the model using test data.
11. Predict the classes of test sequences.
12. Display the actual and predicted classes.
13. Plot the training and validation accuracy.

CODE :-

```
import numpy as np

import tensorflow as tf

import matplotlib.pyplot as plt


X = np.array([

    [1, 2, 3, 4, 5],

    [2, 3, 4, 5, 6],

    [3, 4, 5, 6, 7],

    [4, 5, 6, 7, 8],

    [5, 6, 7, 8, 9],

    [6, 7, 8, 9, 10],

    [7, 8, 9, 10, 11],

    [8, 9, 10, 11, 12],

    [9, 10, 11, 12, 13],

    [10, 11, 12, 13, 14],

    [11, 12, 13, 14, 15],

    [12, 13, 14, 15, 16],

    [13, 14, 15, 16, 17],

    [14, 15, 16, 17, 18],

    [15, 16, 17, 18, 19],

    [16, 17, 18, 19, 20]

])


y = np.array([

    0, 0, 0, 0,

    1, 1, 1, 1,

    0, 0, 0, 0,

    1, 1, 1, 1
```

```
])

X_train = X[:12]
y_train = y[:12]

X_test = X[12:]
y_test = y[12:]

model = tf.keras.Sequential([
    tf.keras.Input(shape=(5,)),
    tf.keras.layers.Embedding(input_dim=30, output_dim=16),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(2, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=2,
    validation_split=0.2,
    verbose=1
)

test_loss, test_accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)
```

```
print("Test Loss:", test_loss)

print("Test Accuracy:", test_accuracy)


predictions = model.predict(X_test, verbose=0)

predicted_classes = predictions.argmax(axis=1)


print("Predicted Classes:", predicted_classes)

print("Actual Classes:", y_test)


plt.figure(figsize=(8, 5))

plt.plot(history.history["accuracy"], label="Training Accuracy")

plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.xlabel("Epochs")

plt.ylabel("Accuracy")

plt.title("RNN Training and Validation Accuracy")

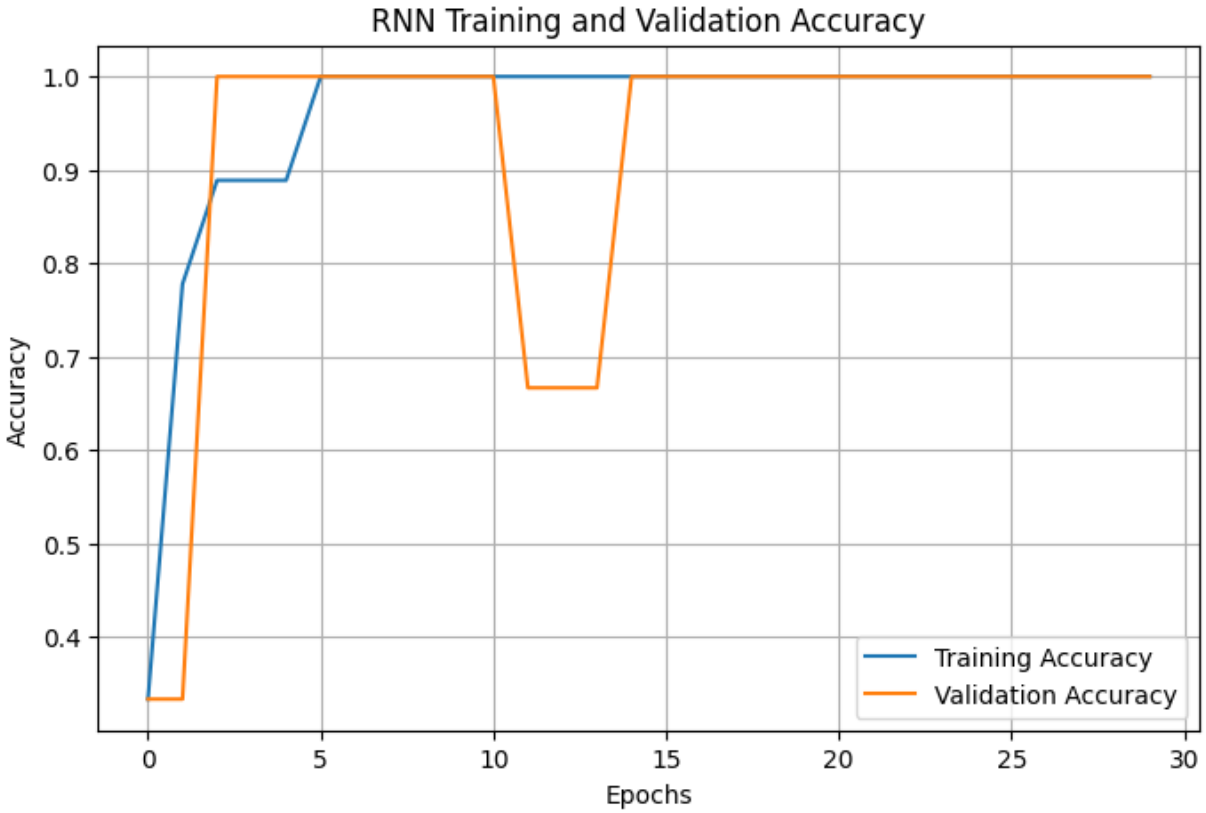
plt.legend()

plt.grid(True)

plt.show()
```

OUTPUT :-

```
Epoch 1/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 1s 69ms/step - accuracy: 0.3333 - loss: 0.7004 - val_accuracy: 0.3333 - val_loss: 0.7060
Epoch 2/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7778 - loss: 0.6652 - val_accuracy: 0.3333 - val_loss: 0.6855
Epoch 3/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8889 - loss: 0.6365 - val_accuracy: 1.0000 - val_loss: 0.6606
Epoch 4/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8889 - loss: 0.6066 - val_accuracy: 1.0000 - val_loss: 0.6315
Epoch 5/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8889 - loss: 0.5733 - val_accuracy: 1.0000 - val_loss: 0.5985
Epoch 6/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 1.0000 - loss: 0.5347 - val_accuracy: 1.0000 - val_loss: 0.5632
Epoch 7/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.4887 - val_accuracy: 1.0000 - val_loss: 0.5284
Epoch 8/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.4333 - val_accuracy: 1.0000 - val_loss: 0.4978
Epoch 9/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 0.3670 - val_accuracy: 1.0000 - val_loss: 0.4735
Epoch 10/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 0.2920 - val_accuracy: 1.0000 - val_loss: 0.4538
Epoch 11/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 0.2166 - val_accuracy: 1.0000 - val_loss: 0.4327
Epoch 12/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 0.1515 - val_accuracy: 0.6667 - val_loss: 0.4027
Epoch 13/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 0.1019 - val_accuracy: 0.6667 - val_loss: 0.3620
Epoch 14/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.0672 - val_accuracy: 0.6667 - val_loss: 0.3173
Epoch 15/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 0.0443 - val_accuracy: 1.0000 - val_loss: 0.2772
Epoch 16/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.0299 - val_accuracy: 1.0000 - val_loss: 0.2457
Epoch 17/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 0.0211 - val_accuracy: 1.0000 - val_loss: 0.2226
Epoch 18/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 1.0000 - loss: 0.0155 - val_accuracy: 1.0000 - val_loss: 0.2063
Epoch 19/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 0.0120 - val_accuracy: 1.0000 - val_loss: 0.1949
Epoch 20/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0096 - val_accuracy: 1.0000 - val_loss: 0.1868
Epoch 21/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0079 - val_accuracy: 1.0000 - val_loss: 0.1809
Epoch 22/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0067 - val_accuracy: 1.0000 - val_loss: 0.1763
Epoch 23/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 1.0000 - loss: 0.0058 - val_accuracy: 1.0000 - val_loss: 0.1727
Epoch 24/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 1.0000 - loss: 0.0051 - val_accuracy: 1.0000 - val_loss: 0.1696
Epoch 25/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 1.0000 - loss: 0.0046 - val_accuracy: 1.0000 - val_loss: 0.1669
Epoch 26/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 1.0000 - loss: 0.0041 - val_accuracy: 1.0000 - val_loss: 0.1644
Epoch 27/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 1.0000 - loss: 0.0038 - val_accuracy: 1.0000 - val_loss: 0.1620
Epoch 28/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 1.0000 - loss: 0.0035 - val_accuracy: 1.0000 - val_loss: 0.1598
Epoch 29/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 1.0000 - loss: 0.0032 - val_accuracy: 1.0000 - val_loss: 0.1577
Epoch 30/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 1.0000 - loss: 0.0030 - val_accuracy: 1.0000 - val_loss: 0.1557
Test Loss: 2.469534158706665
Test Accuracy: 0.0
Predicted Classes: [0 0 0 0]
Actual Classes: [1 1 1 1]
```



CONCLUSION :-

The experiment successfully implemented a Recurrent Neural Network for sequence classification. The RNN processed sequential inputs using its hidden state and learned patterns from the sequence. The trained model classified the input sequences into their respective classes and its performance was evaluated using test accuracy and predicted outputs.